

Recursion, Searching, Sorting, and Callbacks in C Programming

Table of Contents

1. [Introduction to Recursion](#)
2. [Problem Solving using Recursion](#)
3. [Searching Algorithms](#)
4. [Sorting Algorithms](#)
5. [Problem Solving using Callbacks](#)

Introduction to Recursion

Recursion is a programming technique where a function calls itself to solve a problem. The solution to a complex problem is expressed in terms of simpler instances of the same problem.

Key Components of Recursion:

1. **Base Case:** The condition that stops the recursion
2. **Recursive Case:** The part where the function calls itself

Basic Syntax of a Recursive Function in C:

```
c
return_type function_name(parameters) {
    // Base case
    if (base_case_condition) {
        return base_case_value;
    }
    // Recursive case
    else {
        // Process data and make recursive call(s)
        return function_name(modified_parameters);
    }
}
```

Example: Factorial Calculation

c

```
#include <stdio.h>

// Recursive function to calculate factorial
unsigned long factorial(unsigned int n) {
    // Base case
    if (n == 0 || n == 1) {
        return 1;
    }
    // Recursive case
    else {
        return n * factorial(n - 1);
    }
}

int main() {
    unsigned int number = 5;
    printf("Factorial of %u is %lu\n", number, factorial(number));
    return 0;
}
```

Output:

```
Factorial of 5 is 120
```

In this example:

- The base case is when n equals 0 or 1, where the factorial is 1.
- The recursive case calculates $n * \text{factorial}(n-1)$.

Memory Stack During Recursion

Understanding what happens in memory is crucial:

For $\text{factorial}(5)$:

1. Call $\text{factorial}(5)$ → Waits for $\text{factorial}(4)$
2. Call $\text{factorial}(4)$ → Waits for $\text{factorial}(3)$
3. Call $\text{factorial}(3)$ → Waits for $\text{factorial}(2)$
4. Call $\text{factorial}(2)$ → Waits for $\text{factorial}(1)$
5. Call $\text{factorial}(1)$ → Returns 1 (base case)
6. $\text{factorial}(2)$ completes: $2 * 1 = 2$

- 7. `factorial(3)` completes: $3 * 2 = 6$
- 8. `factorial(4)` completes: $4 * 6 = 24$
- 9. `factorial(5)` completes: $5 * 24 = 120$

Recursion vs. Iteration

Recursion often provides a more elegant and natural solution for certain problems, but it comes with a cost:

Aspect	Recursion	Iteration
Memory	Higher (stack space)	Lower
Speed	Generally slower	Generally faster
Clarity	Often clearer for certain problems	Can be more complex for the same problems
Stack Overflow	Risk exists	No risk

Problem Solving using Recursion

Let's explore more complex problems that can be elegantly solved using recursion.

Example 1: Fibonacci Sequence

c

```
#include <stdio.h>

// Recursive function to find the nth Fibonacci number
int fibonacci(int n) {
    // Base cases
    if (n <= 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }

    // Recursive case
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n = 10;
    printf("The %dth Fibonacci number is: %d\n", n, fibonacci(n));

    printf("Fibonacci sequence up to %d:\n", n);
    for (int i = 0; i <= n; i++) {
        printf("%d ", fibonacci(i));
    }
    printf("\n");

    return 0;
}
```

Output:

```
The 10th Fibonacci number is: 55
Fibonacci sequence up to 10:
0 1 1 2 3 5 8 13 21 34 55
```

Note: Simple recursive Fibonacci is inefficient due to redundant calculations. For production code, dynamic programming or memoization should be used.

Example 2: Tower of Hanoi

The Tower of Hanoi is a classic recursion problem:

c

```
#include <stdio.h>

// Recursive function to solve Tower of Hanoi
void towerOfHanoi(int n, char source, char auxiliary, char destination) {
    // Base case
    if (n == 1) {
        printf("Move disk 1 from rod %c to rod %c\n", source, destination);
        return;
    }

    // Recursive cases
    // Move n-1 disks from source to auxiliary using destination as auxiliary
    towerOfHanoi(n - 1, source, destination, auxiliary);

    // Move the nth disk from source to destination
    printf("Move disk %d from rod %c to rod %c\n", n, source, destination);

    // Move n-1 disks from auxiliary to destination using source as auxiliary
    towerOfHanoi(n - 1, auxiliary, source, destination);
}

int main() {
    int n = 3; // Number of disks
    towerOfHanoi(n, 'A', 'B', 'C'); // A=source, B=auxiliary, C=destination
    return 0;
}
```

Output:

```
Move disk 1 from rod A to rod C
Move disk 2 from rod A to rod B
Move disk 1 from rod C to rod B
Move disk 3 from rod A to rod C
Move disk 1 from rod B to rod A
Move disk 2 from rod B to rod C
Move disk 1 from rod A to rod C
```

Example 3: Binary Tree Traversal

Recursion is natural for tree traversals:


```

#include <stdio.h>
#include <stdlib.h>

// Binary tree node structure
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Recursive inorder traversal
void inorderTraversal(struct Node* root) {
    // Base case
    if (root == NULL) {
        return;
    }

    // Recursive cases
    inorderTraversal(root->left); // Traverse Left subtree
    printf("%d ", root->data);    // Visit node
    inorderTraversal(root->right); // Traverse right subtree
}

// Recursive preorder traversal
void preorderTraversal(struct Node* root) {
    if (root == NULL) {
        return;
    }

    printf("%d ", root->data); // Visit node
    preorderTraversal(root->left); // Traverse Left subtree
    preorderTraversal(root->right); // Traverse right subtree
}

// Recursive postorder traversal
void postorderTraversal(struct Node* root) {
    if (root == NULL) {

```

```

        return;
    }

    postorderTraversal(root->left); // Traverse left subtree
    postorderTraversal(root->right); // Traverse right subtree
    printf("%d ", root->data);      // Visit node
}

int main() {
    // Create a sample binary tree
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    printf("Inorder traversal: ");
    inorderTraversal(root);
    printf("\n");

    printf("Preorder traversal: ");
    preorderTraversal(root);
    printf("\n");

    printf("Postorder traversal: ");
    postorderTraversal(root);
    printf("\n");

    // Free memory (not shown in detail)
    // Would need a recursive free function

    return 0;
}

```

Output:

```

Inorder traversal: 4 2 5 1 3
Preorder traversal: 1 2 4 5 3
Postorder traversal: 4 5 2 3 1

```

Searching Algorithms

Searching is the process of finding a specific element within a collection of data. Let's implement the two most common searching algorithms.

Linear Search

Linear search sequentially checks each element until the target is found or the end is reached.


```

#include <stdio.h>

// Recursive Linear search
int recursiveLinearSearch(int arr[], int size, int target, int currentIndex) {
    // Base cases
    if (currentIndex >= size) { // Element not found
        return -1;
    }
    if (arr[currentIndex] == target) { // Element found
        return currentIndex;
    }

    // Recursive case
    return recursiveLinearSearch(arr, size, target, currentIndex + 1);
}

// Iterative Linear search
int iterativeLinearSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Return index of the found element
        }
    }
    return -1; // Element not found
}

int main() {
    int arr[] = {12, 34, 54, 2, 3};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 3;

    // Using recursive Linear search
    int recResult = recursiveLinearSearch(arr, size, target, 0);
    if (recResult != -1) {
        printf("Recursive Linear Search: Element found at index %d\n", recResult);
    } else {
        printf("Recursive Linear Search: Element not found\n");
    }

    // Using iterative Linear search
    int iterResult = iterativeLinearSearch(arr, size, target);
    if (iterResult != -1) {
        printf("Iterative Linear Search: Element found at index %d\n", iterResult);
    } else {
        printf("Iterative Linear Search: Element not found\n");
    }
}

```

```
    return 0;  
}
```

Output:

```
Recursive Linear Search: Element found at index 4  
Iterative Linear Search: Element found at index 4
```

Binary Search

Binary search is much faster than linear search for sorted arrays, with $O(\log n)$ time complexity.


```
#include <stdio.h>

// Recursive binary search
int recursiveBinarySearch(int arr[], int low, int high, int target) {
    // Base case
    if (low > high) {
        return -1; // Element not found
    }

    // Calculate middle index
    int mid = low + (high - low) / 2;

    // Found the element
    if (arr[mid] == target) {
        return mid;
    }

    // Recursive cases
    // Search in the left half
    if (arr[mid] > target) {
        return recursiveBinarySearch(arr, low, mid - 1, target);
    }

    // Search in the right half
    return recursiveBinarySearch(arr, mid + 1, high, target);
}

// Iterative binary search
int iterativeBinarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Check if target is present at mid
        if (arr[mid] == target) {
            return mid;
        }

        // If target is greater, ignore left half
        if (arr[mid] < target) {
            low = mid + 1;
        }

        // If target is smaller, ignore right half
        else {
```

```

        high = mid - 1;
    }
}

return -1; // Element not found
}

int main() {
    int arr[] = {2, 3, 12, 34, 54, 65, 78}; // Must be sorted for binary search
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 34;

    // Using recursive binary search
    int recResult = recursiveBinarySearch(arr, 0, size - 1, target);
    if (recResult != -1) {
        printf("Recursive Binary Search: Element found at index %d\n", recResult);
    } else {
        printf("Recursive Binary Search: Element not found\n");
    }

    // Using iterative binary search
    int iterResult = iterativeBinarySearch(arr, size, target);
    if (iterResult != -1) {
        printf("Iterative Binary Search: Element found at index %d\n", iterResult);
    } else {
        printf("Iterative Binary Search: Element not found\n");
    }

    return 0;
}

```

Output:

```

Recursive Binary Search: Element found at index 3
Iterative Binary Search: Element found at index 3

```

Sorting Algorithms

Sorting arranges elements in a specific order. Let's implement several common sorting algorithms using both recursive and iterative approaches.

Quick Sort (Recursive)

Quick sort is an efficient, divide-and-conquer sorting algorithm with an average time complexity of $O(n \log n)$.


```

#include <stdio.h>

// Function to swap two elements
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to partition the array
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Select the rightmost element as pivot
    int i = (low - 1);      // Index of smaller element

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++; // Increment index of smaller element
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// Recursive QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // pi is the partitioning index
        int pi = partition(arr, low, high);

        // Sort elements before and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {

```

```
int arr[] = {10, 7, 8, 9, 1, 5};
int size = sizeof(arr) / sizeof(arr[0]);

printf("Original array: ");
printArray(arr, size);

quickSort(arr, 0, size - 1);

printf("Sorted array: ");
printArray(arr, size);

return 0;
}
```

Output:

```
Original array: 10 7 8 9 1 5
Sorted array: 1 5 7 8 9 10
```

Merge Sort (Recursive)

Merge sort is a stable, divide-and-conquer algorithm with a time complexity of $O(n \log n)$.


```

#include <stdio.h>
#include <stdlib.h>

// Merge two subarrays of arr[]
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int *L = (int*)malloc(n1 * sizeof(int));
    int *R = (int*)malloc(n2 * sizeof(int));

    // Copy data to temporary arrays
    for (i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
    for (j = 0; j < n2; j++) {
        R[j] = arr[mid + 1 + j];
    }

    // Merge temporary arrays back into arr[left..right]
    i = 0;        // Initial index of first subarray
    j = 0;        // Initial index of second subarray
    k = left;     // Initial index of merged subarray

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy remaining elements of L[] if any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy remaining elements of R[] if any
    while (j < n2) {

```

```

        arr[k] = R[j];
        j++;
        k++;
    }

    // Free the allocated memory
    free(L);
    free(R);
}

// Recursive merge sort function
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        // Find the middle point
        int mid = left + (right - left) / 2;

        // Sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, size);

    mergeSort(arr, 0, size - 1);

    printf("Sorted array: ");
    printArray(arr, size);
}

```

```
    return 0;  
}
```

Output:

Original array: 12 11 13 5 6 7

Sorted array: 5 6 7 11 12 13

Bubble Sort (Recursive)

Bubble sort is a simple comparison-based algorithm. Here's a recursive implementation:


```
#include <stdio.h>
```

```
// A recursive function for bubble sort
```

```
void recursiveBubbleSort(int arr[], int n) {
```

```
    // Base case
```

```
    if (n == 1) {
```

```
        return;
```

```
    }
```

```
    // One pass of bubble sort. After this pass, the largest element is moved to the end
```

```
    int swapped = 0;
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        if (arr[i] > arr[i + 1]) {
```

```
            // Swap arr[i] and arr[i+1]
```

```
            int temp = arr[i];
```

```
            arr[i] = arr[i + 1];
```

```
            arr[i + 1] = temp;
```

```
            swapped = 1;
```

```
        }
```

```
    }
```

```
    // If no swapping occurred in this pass, array is sorted
```

```
    if (swapped == 0) {
```

```
        return;
```

```
    }
```

```
    // Largest element is fixed, recur for remaining array
```

```
    recursiveBubbleSort(arr, n - 1);
```

```
}
```

```
// Function to print an array
```

```
void printArray(int arr[], int size) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main() {
```

```
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
```

```
    int size = sizeof(arr) / sizeof(arr[0]);
```

```
    printf("Original array: ");
```

```
    printArray(arr, size);
```

```
    recursiveBubbleSort(arr, size);
```



```
    printf("Sorted array: ");  
    printArray(arr, size);  
  
    return 0;  
}
```

Output:

```
Original array: 64 34 25 12 22 11 90  
Sorted array: 11 12 22 25 34 64 90
```

Problem Solving using Callbacks

Callbacks are function pointers in C that allow passing functions as arguments to other functions. They're powerful for creating flexible, reusable code.

Understanding Function Pointers

First, let's understand function pointers:

c

```
#include <stdio.h>

// A simple function
int add(int a, int b) {
    return a + b;
}

// Another simple function
int multiply(int a, int b) {
    return a * b;
}

int main() {
    // Declare a function pointer
    int (*operation)(int, int);

    // Assign function address to the pointer
    operation = add;
    printf("Result of addition: %d\n", operation(5, 3));

    // Change the function being pointed to
    operation = multiply;
    printf("Result of multiplication: %d\n", operation(5, 3));

    return 0;
}
```

Output:

```
Result of addition: 8
Result of multiplication: 15
```

Callbacks for Searching

Let's create a generic search function using callbacks:


```

#include <stdio.h>

// A search function that uses a callback to determine if an element matches
int findElement(int arr[], int size, int (*matchFunc)(int, int), int key) {
    for (int i = 0; i < size; i++) {
        if (matchFunc(arr[i], key)) {
            return i; // Return index if element is found
        }
    }
    return -1; // Return -1 if element is not found
}

// Callback for exact match
int exactMatch(int element, int key) {
    return element == key;
}

// Callback for finding elements greater than key
int greaterThan(int element, int key) {
    return element > key;
}

// Callback for finding even elements
int isEven(int element, int key) {
    // Key is ignored in this callback
    return element % 2 == 0;
}

int main() {
    int arr[] = {5, 12, 7, 20, 15, 8};
    int size = sizeof(arr) / sizeof(arr[0]);

    // Find exact match for 7
    int index = findElement(arr, size, exactMatch, 7);
    if (index != -1) {
        printf("Element 7 found at index: %d\n", index);
    } else {
        printf("Element 7 not found\n");
    }

    // Find first element greater than 10
    index = findElement(arr, size, greaterThan, 10);
    if (index != -1) {
        printf("First element greater than 10 found at index %d with value %d\n", index, arr[index]);
    } else {
        printf("No element greater than 10 found\n");
    }
}

```

```

}

// Find first even element
index = findElement(arr, size, isEven, 0); // Key is ignored in isEven
if (index != -1) {
    printf("First even element found at index %d with value %d\n", index, arr[index]);
} else {
    printf("No even element found\n");
}

return 0;
}

```

Output:

```

Element 7 found at index: 2
First element greater than 10 found at index 1 with value 12
First even element found at index 1 with value 12

```

Callbacks for Sorting

We can use callbacks to create a generic sorting function:


```
#include <stdio.h>
```

```
// Function to swap two integers
```

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
// A generic bubble sort function that uses a callback to determine sort order
```

```
void bubbleSort(int arr[], int size, int (*compare)(int, int)) {  
    for (int i = 0; i < size - 1; i++) {  
        for (int j = 0; j < size - i - 1; j++) {  
            // If compare function returns true, swap the elements  
            if (compare(arr[j], arr[j + 1])) {  
                swap(&arr[j], &arr[j + 1]);  
            }  
        }  
    }  
}
```

```
// Callback for ascending order
```

```
int ascending(int a, int b) {  
    return a > b;  
}
```

```
// Callback for descending order
```

```
int descending(int a, int b) {  
    return a < b;  
}
```

```
// Callback for even-first ordering (even numbers before odd numbers)
```

```
int evenFirst(int a, int b) {  
    // If a is odd and b is even, swap them  
    if (a % 2 != 0 && b % 2 == 0) {  
        return 1;  
    }  
    // If both are even or both are odd, maintain original order  
    else if ((a % 2 == 0 && b % 2 == 0) || (a % 2 != 0 && b % 2 != 0)) {  
        return a > b; // Sort in ascending order within the same parity  
    }  
    // If a is even and b is odd, keep them as is  
    return 0;  
}
```

```
// Function to print an array
```

```

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = sizeof(arr) / sizeof(arr[0]);
    int arrCopy1[size], arrCopy2[size];

    // Copy original array to preserve it for multiple sorts
    for (int i = 0; i < size; i++) {
        arrCopy1[i] = arr[i];
        arrCopy2[i] = arr[i];
    }

    printf("Original array: ");
    printArray(arr, size);

    // Sort in ascending order
    bubbleSort(arr, size, ascending);
    printf("Sorted in ascending order: ");
    printArray(arr, size);

    // Sort in descending order
    bubbleSort(arrCopy1, size, descending);
    printf("Sorted in descending order: ");
    printArray(arrCopy1, size);

    // Sort by even-first rule
    bubbleSort(arrCopy2, size, evenFirst);
    printf("Sorted with even numbers first: ");
    printArray(arrCopy2, size);

    return 0;
}

```

Output:

```

Original array: 64 34 25 12 22 11 90
Sorted in ascending order: 11 12 22 25 34 64 90
Sorted in descending order: 90 64 34 25 22 12 11
Sorted with even numbers first: 12 22 34 64 90 11 25

```


Callbacks for Counting and Filtering

Callbacks can be used for various data processing tasks:


```

#include <stdio.h>
#include <stdlib.h>

// Count elements in array that match a certain condition
int countElements(int arr[], int size, int (*condition)(int)) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (condition(arr[i])) {
            count++;
        }
    }
    return count;
}

// Filter elements into a new array based on a condition
int* filterElements(int arr[], int size, int (*condition)(int), int* resultSize) {
    // First, count how many elements will pass the filter
    *resultSize = countElements(arr, size, condition);

    // Allocate memory for the result array
    int* result = (int*)malloc(*resultSize * sizeof(int));

    // Copy elements that match the condition
    int j = 0;
    for (int i = 0; i < size; i++) {
        if (condition(arr[i])) {
            result[j++] = arr[i];
        }
    }

    return result;
}

// Condition functions
int isEven(int num) {
    return num % 2 == 0;
}

int isOdd(int num) {
    return num % 2 != 0;
}

int isPositive(int num) {
    return num > 0;
}

```

```

int isNegative(int num) {
    return num < 0;
}

int isPrime(int num) {
    if (num <= 1) return 0;
    if (num <= 3) return 1;
    if (num % 2 == 0 || num % 3 == 0) return 0;

    for (int i = 5; i * i <= num; i += 6) {
        if (num % i == 0 || num % (i + 2) == 0) {
            return 0;
        }
    }
    return 1;
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {-3, 2, 0, 7, -5, 11, 4, 6, -8, 9};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, size);

    // Count even numbers
    printf("Number of even elements: %d\n", countElements(arr, size, isEven));

    // Count positive numbers
    printf("Number of positive elements: %d\n", countElements(arr, size, isPositive));

    // Count prime numbers
    printf("Number of prime elements: %d\n", countElements(arr, size, isPrime));

    // Filter even numbers
    int evenSize;
    int* evenNumbers = filterElements(arr, size, isEven, &evenSize);
    printf("Even elements: ");
    printArray(evenNumbers, evenSize);
}

```

```

// Filter prime numbers
int primeSize;
int* primeNumbers = filterElements(arr, size, isPrime, &primeSize);
printf("Prime elements: ");
printArray(primeNumbers, primeSize);

// Free allocated memory
free(evenNumbers);
free(primeNumbers);

return 0;
}

```

Output:

```

Original array: -3 2 0 7 -5 11 4 6 -8 9
Number of even elements: 5
Number of positive elements: 6
Number of prime elements: 4
Even elements: 2 0 4 6 -8
Prime elements: 2 7 11

```

Advanced Example: Map-Reduce with Callbacks

Let's implement a simple Map-Reduce pattern using callbacks:

```

c

#include

```